

---

# Font to Binary Converter

## プログラム設計仕様書

ドットフォント→バイナリ配列変換 Web ツール

---

|          |                          |
|----------|--------------------------|
| プロジェクト   | Font to Binary Converter |
| ドキュメント種別 | プログラム設計仕様書               |
| 版数       | Rev. 1.3                 |
| 発行日      | 2026 年 4 月 21 日          |
| 著者       | Shiozawa                 |

本書は Font to Binary Converter の設計仕様を記述する。

## 改訂履歴

| 版   | 日付         | 著者       | 変更内容                                                                                                                                                                            |
|-----|------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0 | 2026-04-21 | Shiozawa | 初版作成。Python 版から Web (React + Vite + TypeScript) 実装へ全面刷新。                                                                                                                        |
| 1.1 | 2026-04-21 | Shiozawa | UX 改善。かんたん／詳細の 2 モード切替、用途別プリセット (Arduino/C 2D/Python/MicroPython/JSON/記号アート)、ドットサイズのクイックボタン (8/12/16/24/32) を追加。GitHub リンクのリポジトリ URL 修正。                                        |
| 1.2 | 2026-04-21 | Shiozawa | レイアウト刷新 (上段プレビュー帯 + 下段 2 カラム)。プレビューを横スクロール形式に変更しズーム倍率を追加。プレビュー/エディタ/自由キャンバスに <b>N ドット毎の強調補助線</b> 機能を追加。新モード「 <b>自由</b> 」を追加し、任意 W×H のスクロール可能なキャンバスに自由描画した結果をそのまま出力フォーマットに変換可能に。 |
| 1.3 | 2026-04-21 | Shiozawa | 公開前の初期状態調整。文字入力系のデフォルト文字列を "ABC あいう" に、プレビュー補助線のデフォルトを「なし」に変更。自由キャンバスの初期サイズを 64×8 (8×8 のグリフが 8 文字並ぶ想定、補助線 8 ドット) へ変更。                                                           |

### ▷ 旧版との関係

本書の Rev. 1.0 は、旧来の Python/Tkinter 実装 (font\_to\_binary\_GUI.py および font\_to\_binary\_CUI.py) を置き換えた Web 実装の仕様である。旧ソースは legacy/ ディレクトリにそのまま残されている。

# 目次

|                           |    |
|---------------------------|----|
| 改訂履歴                      | 1  |
| 1 概要                      | 4  |
| 1.1 主な特徴                  | 4  |
| 1.2 動作環境                  | 5  |
| 2 背景と課題                   | 6  |
| 2.1 旧 Python/Tkinter 版の課題 | 6  |
| 2.2 組み込み現場での典型的な需要        | 6  |
| 2.3 Web アプリ化の動機           | 6  |
| 3 目的                      | 7  |
| 3.1 達成基準                  | 7  |
| 3.2 非目標                   | 7  |
| 4 アーキテクチャ                 | 8  |
| 4.1 全体構成                  | 8  |
| 4.2 レイヤ分割                 | 8  |
| 4.3 データフロー                | 8  |
| 4.4 画面レイアウト               | 8  |
| 4.5 UI モード切替              | 9  |
| 4.6 ビルド・配布パイプライン          | 9  |
| 5 ファイル構成                  | 10 |
| 5.1 ディレクトリツリー             | 10 |
| 5.2 core 層                | 11 |
| 5.3 hooks 層               | 11 |
| 5.4 UI 層                  | 12 |
| 5.5 その他                   | 12 |
| 6 技術スタック                  | 13 |
| 6.1 選定方針                  | 13 |
| 6.2 使用ライブラリ・ツール           | 13 |
| 6.3 ブラウザ API              | 13 |
| 6.4 同梱フォント                | 13 |
| 6.5 ビルド設定                 | 14 |
| 7 API 仕様                  | 15 |
| 7.1 型定義 (types.ts)        | 15 |
| 7.2 ラスタライズ (rasterize.ts) | 16 |
| 7.3 エンコード (encode.ts)     | 17 |
| 7.4 フォーマット (format.ts)    | 17 |
| 7.5 フォント管理 (fonts.ts)     | 17 |

|     |                  |    |
|-----|------------------|----|
| 7.6 | 共有リンク (share.ts) | 18 |
| 7.7 | React Hooks      | 18 |
| 8   | <b>実行モデル</b>     | 19 |
| 8.1 | 起動シーケンス          | 19 |
| 8.2 | ラストライズの詳細        | 19 |
| 8.3 | ビットパック規則         | 19 |
| 8.4 | 状態遷移 (Reducer)   | 19 |
| 8.5 | 永続化              | 20 |
| 8.6 | 共有リンクのラウンドトリップ   | 20 |
| 8.7 | パフォーマンス考慮        | 20 |
| 8.8 | デプロイ             | 20 |

# 1 概要

---

Font to Binary Converter は、ドットフォントで描画された文字を二次元バイナリ配列 (0/1) または各種エンコード形式に変換し、C/C++/Python/JavaScript 等のソースコードに直接埋め込める形式で出力する Web ツールである。

本ツールは以下の用途を主な想定としている。

- マイコン (Arduino, ESP32, RZ/A1 等) の 0.96" OLED、LCD 等小型ディスプレイ上へのフォント描画用ビットマップデータ生成。
- LED マトリクスパネル (8 × 8, 16 × 16, 32 × 32) の文字表示用データ。
- ゲーム・デモ用の固定幅ドットフォント素材生成。
- ピクセルアート作成時の下書きテンプレート。

## 1.1 主な特徴

- **ブラウザ単体で動作**: サーバー不要。GitHub Pages で静的配信。
- **3 モード UI**: 「かんたん」は 3 ステップ (文字/見た目/用途) で初心者でも迷わず変換可能。「詳細」は全設定を直接編集可能。「自由」は任意 W×H のキャンバスへ自由にドット絵を描画しそのまま出力フォーマットに変換可能。モード選択は LocalStorage に自動保存。
- **2 段レイアウト**: 上段にプレビュー (または自由キャンバス) を横幅いっぱい配置し、下段を「設定 (左) / 出力コード (右)」の 2 カラムに分割。大きなグリフ列や高解像度キャンバスを横スクロールで確認しやすい構成。
- **補助線 (ブロック強調)**: プレビュー、ピクセルエディタ、自由キャンバスのいずれにおいても、**N ドット毎** (なし / 2 / 4 / 8 / 16 / 32px から選択) に太めの強調線を描画し、ブロック境界を視認しやすくする。
- **用途別プリセット**: Arduino/C 言語, C 言語 2D, Python リスト, MicroPython bytes, JSON, 記号アートの 6 種を 1 クリック切替。
- **フォント同梱**: DotGothic16 (16 × 16) と Misaki Gothic 2nd (8 × 8) を同梱。
- **ユーザーフォント対応**: TTF/OTF/WOFF をその場でアップロード可能。
- **多言語出力**: C, C++, Arduino(PROGMEM), Python, JavaScript, TypeScript, Rust, Go, JSON, Markdown, プレーンテキスト, 記号アート。
- **詳細な出力設定**: 2D/3D/フラット/ビットパック構造、10/16/2 進表記、MSB/LSB、行/列優先、0/1 反転など。
- **ピクセルエディタ**: 生成結果を 1 ドット単位で手動編集可能。
- **自由キャンバス**: 1~256 px 角の任意サイズで自由にドット絵を描画可能。ペン/消しゴム/トグル、全体反転、左右・上下反転、90 度回転 (正方形のみ) を備え、補助線とズームでドット単位の精緻な編集に対応。
- **設定の自動保存と共有**: LocalStorage へ自動保存、URL クエリパラメータによる共有リンク生成。

## 1.2 動作環境

| 項目           | 要件                                        |
|--------------|-------------------------------------------|
| ブラウザ         | Chrome / Edge / Firefox / Safari の最新版     |
| JavaScript   | 有効                                        |
| Canvas API   | CanvasRenderingContext2D, getImageData 対応 |
| FontFace API | カスタムフォント読込に使用（主要ブラウザで対応済み）                |
| LocalStorage | 設定の永続化に使用（オプトアウト可）                        |

## 2 背景と課題

---

### 2.1 旧 Python/Tkinter 版の課題

旧版 (`font_to_binary_GUI.py` / `font_to_binary_CUI.py`) は Python + Tkinter で実装された小規模ユーティリティであった。一定の機能は果たしていたが、以下の課題があった。

- **配布の煩雑さ**: Python と関連ライブラリ (Pillow, tkinter) を利用者がインストールする必要があった。
- **UI/UX の制約**: Tkinter のネイティブウィジェットでは近年の Web アプリのような細やかな表現ができず、レイアウトやテーマ切替も貧弱であった。
- **出力形式の限定**: C 配列と一部の表記のみ対応しており、言語切替、ビットパック、MSB/LSB、0/1 反転などの細かな設定ができなかった。
- **ピクセル編集不可**: 生成後の手動補正ができず、細かな調整をしたい場合は結果をコピーして手で書き換える必要があった。
- **共有性**: 設定を他人に渡す手段がなく、再現実験や共同作業に不向きであった。

### 2.2 組み込み現場での典型的な需要

マイコン用ディスプレイ (OLED/LCD/LED マトリクス) へのドットフォント描画では、以下が繰り返し要求される。

- ターゲットマイコン・表示ドライバごとに異なるバイト並び (行方向/列方向、MSB/LSB) への対応。
- FLASH 節約のためのビットパック配列出力 (1bit/pixel)。
- `PROGMEM`, `const`, `static` 等の言語・処理系依存修飾子の付与。
- デザインを確認するためのプレビューと、生成結果の部分修正。

これらを毎回手作業で行うのは非効率であり、「設定を GUI で触り、結果をコピーするだけで済むツール」が常に求められていた。

### 2.3 Web アプリ化の動機

これらの課題に対し、クロスプラットフォームで即起動でき、機能拡張が容易で、URL で設定を共有できるという要求を満たすため、Web アプリケーションとしての再実装を選択した。

- 利用者はブラウザを開くだけで利用可能 (インストール不要)。
- GitHub Pages による無料ホスティングで配布コスト 0。
- React + TypeScript により型安全に機能拡張が可能。
- Canvas API + FontFace API によりフォントラスタライズをサーバレスで実行可能。

### 3 目的

本ツールの目的は次の 3 点に集約される。

1. **即時性**: ブラウザを開いてから配列を取得するまでの手数を最小化する。インストール、ログイン、ファイル入出力を要しない。
2. **網羅性**: 組み込みで実際に遭遇する出力形式（言語・構造・ビット順・エンディアン・修飾子）を広くカバーする。
3. **調整可能性**: 自動生成された結果を 1 ドット単位で手動編集できる手段を提供する。

#### 3.1 達成基準

| 観点    | 基準                                                               |
|-------|------------------------------------------------------------------|
| 起動性   | ブラウザで URL を開くだけで利用可能、認証不要                                        |
| サイズ   | 初期バンドル < 500 KB gzipped（目標）                                      |
| 応答性   | 文字数 100 以下で描画再計算が < 100ms（目標）                                    |
| 対応文字  | Unicode 基本多言語面（BMP）全域、16 × 16 の想定でのみ品質保証                         |
| 多言語出力 | C/C++/Arduino/Python/JS/TS/Rust/Go/JSON/MD/Plain/Symbols の 12 形式 |
| 出力構造  | 2 次元/3 次元/フラット/ビットパック（行・列）の 5 種類                                 |
| 編集機能  | ペン/消しゴム/反転/回転/クリア/トグル                                            |
| 永続化   | LocalStorage 自動保存、URL 共有可能、オプトアウト可                               |

#### 3.2 非目標

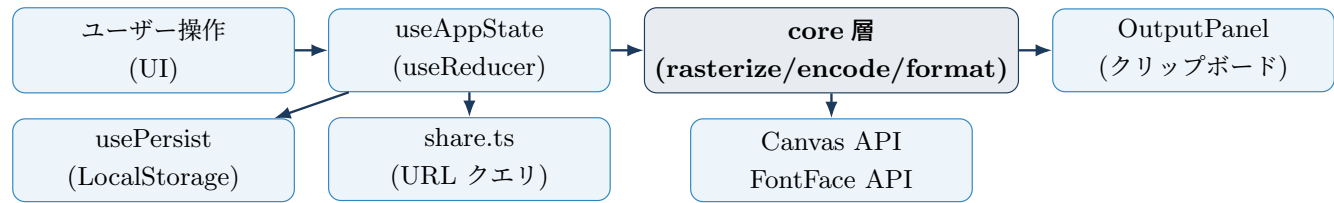
以下は本ツールのスコープ外とする。

- アンチエイリアス付きのグレースケール出力（本ツールは 2 値ラスタライズに特化）。
- SVG や PNG などの画像ファイル形式での直接出力（必要ならプレビューを外部でキャプチャする）。
- ベクトルフォントの編集・グリフ追加機能。
- サーバーサイドでのフォント処理（すべてクライアントサイドで完結）。
- 商用フォントの再配布（同梱フォントはライセンスが明確なもののみ）。

## 4    アーキテクチャ

### 4.1    全体構成

本ツールは純粋なクライアントサイド Single Page Application (SPA) であり、サーバー通信は行わない（フォントファイル取得を除く）。



### 4.2    レイヤ分割

| レイヤ                | 責務                                                                                   |
|--------------------|--------------------------------------------------------------------------------------|
| src/core/          | 純粋関数群。DOM 依存を最小化（Canvas 利用箇所のみ）。ラスタライズ、ビットパック、フォーマッタ、共有 URL エンコード。                   |
| src/hooks/         | React フック。状態管理（useReducer）、永続化（LocalStorage）、テーマ切替、グリフメモ化。                           |
| src/components/    | UI コンポーネント。shadcn/ui プリミティブを基盤とし、パネル単位で責務分離。                                         |
| src/components/ui/ | Radix UI ベースの汎用プリミティブ（Button, Card, Input, Tabs, Switch, Slider, Select, Tooltip 等）。 |

### 4.3    データフロー

- ユーザー入力 → useAppState の dispatch でアクション発行。
- Reducer が新しい AppState を返し、コンポーネントが再描画。
- useGlyphs が入力テキストとフォント設定に基づき rasterizeChar() を再実行しグリフ配列をメモ化。
- formatOutput() がグリフ配列 + フォーマット設定から出力文字列を生成し、OutputPanel が表示。
- 同時に usePersist が AppState を LocalStorage に書き戻す（overrides を除く）。

### 4.4    画面レイアウト

画面は上段と下段の 2 セクションで構成される。上段はプレビュー（または自由キャンバス）を幅いっぱい配置し、横スクロールで大きな文字列や高解像度キャンバスを一望できるようにする。下段は 12 カラムグリッドの 5/7 分割で、左に設定パネル、右に生成コードパネルを並べる。

### 4.5 UI モード切替

初心者・上級者・自由描画という 3 つのユースケースをカバーするため、上段コンテンツと下段の設定カラムのコンテンツを `useMode` (`font_to_bin_mode_v1` キーで `LocalStorage` 永続化) の値で切り替える。

| モード      | 上段                          | 下段・設定カラム                               |
|----------|-----------------------------|----------------------------------------|
| easy     | PreviewPanel (横スクロール)       | EasyPanel (3 ステップ: 文字 → 見た目 → 用途プリセット) |
| advanced | PreviewPanel (横スクロール)       | InputPanel + FormatPanel (全設定)         |
| free     | FreeCanvasPanel (自由描画キャンバス) | FormatPanel (出力設定のみ)                   |

`free` モードでは、入力テキストに基づく `useGlyphs` の結果ではなく、`AppState.free` が持つ単一マトリックスを 1 枚のグリフとして `OutputPanel` に渡す。これにより、テキスト由来のビットマップとハンドドローのビットマップが同じ出力フォーマッタを共有する。

プリセット選択時は `dispatch({type:"setFormat", format: preset.format})` で `FormatOptions` を部分マージし、詳細モードへ切り替えた後もその設定が継続する。逆に詳細モードで変更した内容がどのプリセットにも一致しなくなった場合は、`detectPreset()` が `null` を返し、かんたんモード側では「どのプリセットとも一致していません」旨の案内を表示する。

### 4.6 ビルド・配布パイプライン

- 開発: `npm run dev` (Vite 開発サーバ、HMR 有効)。
- 本番ビルド: `npm run build` (Vite が `dist/` を生成)。
- 自動デプロイ: `main` ブランチ `push` を契機に `.github/workflows/deploy.yml` が起動、GitHub Pages (`gh-pages` 相当の Actions 経由) へ配信。
- SPA フォールバック: デプロイ時に `404.html` を `index.html` と同内容でコピーし、任意パスでのアクセスに対応。

## 5 ファイル構成

### 5.1 ディレクトリツリー

```
font-to-bin/
├── public/
│   ├── fonts/
│   │   ├── DotGothic16-Regular.ttf
│   │   └── misaki_gothic_2nd.ttf
│   ├── favicon.svg
│   └── manifest.webmanifest
├── src/
│   ├── core/
│   │   ├── types.ts          ... 型定義 (AppState, Glyph, Format 他)
│   │   ├── fonts.ts          ... 同梱フォント定義 + カスタム登録API
│   │   ├── rasterize.ts      ... Canvas によるグリフ2値化
│   │   ├── encode.ts         ... ビットパック・数値フォーマット
│   │   ├── format.ts         ... 言語別出力文字列生成
│   │   ├── share.ts          ... URLクエリパラメータ共有
│   │   ├── presets.ts        ... 用途別プリセット (FormatOptions 雛形)
│   │   └── defaults.ts       ... 既定値
│   ├── components/
│   │   ├── ui/               ... shadcn/ui プリミティブ群
│   │   ├── Header.tsx
│   │   ├── EasyPanel.tsx     ... かんたんモード (3ステップ)
│   │   ├── InputPanel.tsx
│   │   ├── FormatPanel.tsx
│   │   ├── PreviewPanel.tsx  ... 横スクロール帯プレビュー
│   │   ├── GlyphPreview.tsx  ... 補助線対応のグリフ描画
│   │   ├── OutputPanel.tsx
│   │   ├── PixelEditor.tsx
│   │   └── FreeCanvasPanel.tsx ... 自由モードのキャンバス
│   ├── hooks/
│   │   ├── useAppState.ts    ... useReducer ラップ
│   │   ├── useGlyphs.ts      ... メモ化ラスタライザ
│   │   ├── usePersist.ts     ... LocalStorage 自動保存
│   │   ├── useMode.ts        ... かんたん/詳細モード切替
│   │   └── useTheme.ts       ... ダーク/ライト
│   ├── lib/
│   │   └── utils.ts          ... cn() ヘルパ
│   ├── App.tsx              ... 3カラムレイアウト + モーダル制御
│   ├── main.tsx             ... エントリーポイント
│   └── index.css             ... Tailwind ベースCSS
├── .github/workflows/deploy.yml ... GitHub Pages 自動デプロイ
├── doc/                     ... LaTeX 設計仕様書
├── legacy/                  ... 旧Python版 (参考保存)
├── index.html
├── package.json
├── tsconfig.json
├── tsconfig.node.json
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
└── README.md
```

## 5.2 core 層

- `src/core/types.ts` — `AppState`, `Glyph`, `FormatOptions` など全型定義。UI と core の契約。
- `src/core/fonts.ts` — `BUILTIN_FONTS` 配列、`loadFont`, `registerCustomFont`。FontFace API でカスタム TTF/OTF を登録。
- `src/core/rasterize.ts` — `rasterizeChar(char, opts) → Glyph`。Canvas へ 1 文字描画し `getImageData` でアルファ値を閾値 2 値化。
- `src/core/encode.ts` — `packRow` / `packColumn` (MSB/LSB ビットパック)、`formatNumber` (bin/hex/dec)。
- `src/core/format.ts` — `formatOutput`。出力言語に応じて `formatC` / `formatPython` / `formatJS` / `formatRust` / `formatGo` / `formatJSON` / `formatMarkdown` / `formatSymbols` / `formatPlain` を呼び分け。
- `src/core/share.ts` — `encodeStateToURL` / `decodeStateFromURL`。AppState を base64url 化して `?s=...` に載せる。
- `src/core/presets.ts` — `PRESETS` 配列 (Arduino/C 言語, C 言語 2D, Python リスト, MicroPython bytes, JSON, 記号アートの 6 種) と `detectPreset()` (現在の `FormatOptions` から該当プリセット ID を推定)。かんたんモードでワンタッチ切替に使用。
- `src/core/defaults.ts` — `DEFAULT_STATE` (`text="ABC あいう"`, `fontId=dotgothic16`, `16×16`, `threshold=128`, `previewGridStep=0` (補助線なし))。自由キャンバス初期値は `64×8` (`8×8` が 8 文字並ぶ想定), `gridStep=8`。

## 5.3 hooks 層

- `src/hooks/useAppState.ts` — `useReducer`。Action は `setText`, `setSize`, `setFormat`, `setOverride` のほか、プレビュー補助線用 `setPreviewGridStep`、自由モード用 `setFreeSize` / `setFreeMatrix` / `setFreePixel` / `setFreeGridStep` / `clearFree` を提供する。
- `src/hooks/useGlyphs.ts` — `text` / `font` / `size` 変更時のみ再ラスタライズするメモ化。overrides をマージ。
- `src/hooks/usePersist.ts` — LocalStorage キー `font-to-bin.state.v1`。
- `src/hooks/useMode.ts` — かんたん/詳細/自由モード切替。LocalStorage キー `font_to_bin_mode.v1`。値は `easy` / `advanced` / `free` のいずれか。
- `src/hooks/useTheme.ts` — `light` / `dark` 切替、`font-to-bin.theme` キー。

## 5.4 UI 層

- `src/components/Header.tsx` — タイトル、**かんたん/詳細モード切替**のセグメントタブ、テーマ切替、URL 共有、GitHub リンク。
- `src/components/EasyPanel.tsx` — かんたんモード用の 3 ステップ UI。**ステップ 1**: 変換したい文字。**ステップ 2**: フォント + ドットサイズクイックボタン (8/12/16/24/32px) + 太字トグル。**ステップ 3**: 用途プリセットを 6 種から 1 クリック選択 (`presets.ts` の PRESETS を使用)。
- `src/components/InputPanel.tsx` — 詳細モードの左上。テキスト、フォント選択、幅/高さ、描画オフセット、太字、閾値。
- `src/components/FormatPanel.tsx` — 出力言語、構造、基数、MSB/LSB、0/1 反転、変数名、データ型。
- `src/components/PreviewPanel.tsx` — グリフ一覧を横スクロール帯で表示。補助線間隔セクタ、ズーム (50–400%) を備える。文字クリックでピクセルエディタを開く。
- `src/components/GlyphPreview.tsx` — 単一グリフの Canvas タイル描画。`gridStep` プロパティで N ドット毎の強調補助線を描画。
- `src/components/OutputPanel.tsx` — 生成結果の表示 + コピー + ダウンロード。
- `src/components/PixelEditor.tsx` — モーダル。ペン/消しゴム/トグル/反転/回転/クリア/ズームに加え、補助線間隔の切替に対応。
- `src/components/FreeCanvasPanel.tsx` — 自由モード専用。W×H(1–256) 指定、スクロール可能なキャンバス、補助線、ズーム、ペン/消しゴム/トグル、反転・回転・全消去。編集結果は `AppState.free.matrix` に `setFreePixel` (1 セル単位) で反映され、出力側では単一グリフとして扱う。
- `src/components/ui/` — Radix UI ベースの shadcn/ui 汎用プリミティブ群。

## 5.5 その他

- `.github/workflows/deploy.yml` — Node 20 で `npm ci → build → Pages` にアップロード。
- `public/fonts/*.ttf` — 同梱フォント (DotGothic16, Misaki Gothic 2nd)。
- `legacy/` — 旧 Python/Tkinter 実装 (参照用、ビルドからは除外)。

## 6 技術スタック

### 6.1 選定方針

- **型安全と開発体験**: TypeScript + Vite による高速な HMR と厳格な型チェック。
- **標準準拠**: Canvas 2D / FontFace / LocalStorage など W3C 標準 API のみを使用し、フレームワーク依存を最小化。
- **UI 構築速度**: Tailwind CSS + shadcn/ui (Radix UI ベース) でスタイルと挙動を分離しつつ、アクセシビリティを確保。
- **ゼロ運用コスト**: GitHub Pages による静的配信。Actions で自動デプロイ、サーバは持たない。

### 6.2 使用ライブラリ・ツール

| 分類        | 名称                       | 用途                            |
|-----------|--------------------------|-------------------------------|
| 言語        | TypeScript 5             | 型安全な JavaScript。              |
| ランタイム     | Node.js 20+              | 開発・ビルド環境。                     |
| ビルド       | Vite 6                   | 開発サーバ + 本番バンドル。               |
| UI 基盤     | React 18                 | 宣言的 UI、Hooks ベース状態管理。         |
| スタイル      | Tailwind CSS 3           | ユーティリティファースト CSS。             |
| UI 部品     | shadcn/ui                | Radix UI + Tailwind プリミティブ集。  |
| UI プリミティブ | Radix UI                 | アクセシブルな Tabs/Select/Slider 等。 |
| アイコン      | lucide-react             | SVG アイコン。                     |
| CVA       | class-variance-authority | バリエーション管理。                    |

### 6.3 ブラウザ API

| API                                | 役割                        |
|------------------------------------|---------------------------|
| Canvas 2D (getContext("2d"))       | フォント描画とピクセル抽出。            |
| getImageData                       | RGBA 配列取得、アルファ閾値 2 値化の基礎。 |
| FontFace                           | カスタム TTF/OTF/WOFF の動的登録。  |
| URL / URLSearchParams              | 共有リンクのクエリパラメータ読み書き。       |
| LocalStorage                       | 設定の永続化。                   |
| navigator.clipboard                | 出力結果のコピー。                 |
| matchMedia("prefers-color-scheme") | OS テーマ検出。                 |

### 6.4 同梱フォント

| フォント              | 推奨サイズ | ライセンス        |
|-------------------|-------|--------------|
| DotGothic16       | 16×16 | SIL OFL 1.1  |
| Misaki Gothic 2nd | 8×8   | 自由配布（商用含む）   |
| システムモノスペース        | 任意    | ブラウザ標準フォント依存 |

## 6.5 ビルド設定

- vite.config.ts: base を本番時のみ/font\_to\_bin/ に切り替え。
- tsconfig.json: strict: true, noUnusedLocals, noUnusedParameters を有効化。
- tailwind.config.js: darkMode: "class"、カスタムカラートークンを定義。
- PWA manifest (public/manifest.webmanifest): アイコン・テーマカラー・表示モードを定義（SW は持たない）。

## 7 API 仕様

本章では `src/core/` が提供する公開 API と主要な型を示す。UI 層はこれらの関数・型にのみ依存し、DOM 操作は `core` 内の `rasterize.ts` に閉じ込められている。

### 7.1 型定義 (types.ts)

#### AppState

AppState は UI 全体の単一情報源であり、永続化・共有の単位でもある。

```
1 type AppState = {
2   text: string;
3   fontId: string;
4   width: number;           // グリフ幅 [px]
5   height: number;          // グリフ高さ [px]
6   size: number;            // 描画フォントサイズ [px]
7   threshold: number;       // 2値化閾値 0-255
8   offsetX: number;
9   offsetY: number;
10  bold: boolean;
11  format: FormatOptions;
12  overrides: Record<number, Matrix>; // charIndex→手動編集結果
13  previewGridStep: number;          // プレビュー補助線間隔(0で無効)
14  free: FreeCanvasState;            // 自由モードのキャンバス
15 };
```

#### FreeCanvasState

自由モードで編集集中のキャンバス状態。1-256 px 角までサポートする。

```
1 type FreeCanvasState = {
2   width: number;           // 1..256
3   height: number;          // 1..256
4   matrix: Matrix;          // matrix[row][col] の Bit 配列
5   gridStep: number;        // Nドット毎の強調補助線 (0で無効)
6 };
```

#### Glyph / Matrix

```
1 type Bit = 0 | 1;
2 type Matrix = Bit[][]; // matrix[row][col]
3 type Glyph = {
4   char: string;
5   width: number;
6   height: number;
7   matrix: Matrix;
8 };
```

## FormatOptions

```
1 type FormatOptions = {
2   language: OutputLanguage; // c|cpp|arduino|python|...
3   structure: Structure;      // matrix2d|matrix3d|flat|bitpack-row|bitpack-
    col
4   dataType: string;          // "uint8_t" 等
5   bitOrder: BitOrder;        // "msb" | "lsb"
6   radix: Radix;              // "bin" | "hex" | "dec"
7   invert: boolean;
8   variableName: string;
9   itemsPerLine: number;
10  includeCharComment: boolean;
11  includeAsciiArt: boolean;
12  padByte: boolean;
13  indent: string;
14  prefix: string;
15  suffix: string;
16 };
```

## 7.2 ラスタライズ (rasterize.ts)

◆ rasterizeChar(char: string, opts: RasterizeOptions): Glyph

指定文字を Canvas に描画し、各ピクセルのアルファ値を opts.threshold で 2 値化して Glyph を返す。

### 引数

- char: 対象文字 (1 文字)。
- opts.fontFamily: CSS の font-family。
- opts.width, opts.height: 出力行列のサイズ。
- opts.size: 描画フォントの px サイズ。
- opts.threshold: [0, 255] のアルファ閾値。
- opts.offsetX, opts.offsetY: 描画オフセット。
- opts.bold: true で bold 書体。

### 戻り値

Glyph。matrix[row][col] は 0/1 の Bit。

## 7.3 エンコード (encode.ts)

◆ `packRow(matrix: Matrix, bitOrder: BitOrder, padByte: boolean): number[]`

`matrix` を行方向に走査して 1bit/pixel にパックする。`padByte=true` の場合、各行を 8bit 境界にゼロ埋めする。

◆ `packColumn(matrix: Matrix, bitOrder: BitOrder, padByte: boolean): number[]`

列方向パック版。OLED/LCD の「縦 8 ドット=1 バイト」フォーマットに相当。

◆ `formatNumber(n: number, radix: Radix, bitWidth: number): string`

`n` を指定基数で文字列化する。`hex` は 0x 接頭辞、`bin` は 0b 接頭辞、`dec` は素の 10 進。`bitWidth` に応じて 0 埋めする。

## 7.4 フォーマット (format.ts)

◆ `formatOutput(glyphs: Glyph[], opts: FormatOptions): string`

グリフ列とフォーマット設定から、ユーザーに表示する最終的な出力文字列を生成する。言語別に以下のヘルパメソッドをディスパッチする。

### ディスパッチ先

- `formatC` (C / C++ / Arduino)
- `formatPython`
- `formatJS` (JavaScript / TypeScript)
- `formatRust`
- `formatGo`
- `formatJSON`
- `formatMarkdown`
- `formatSymbols` (記号アート、■/□)
- `formatPlain` (生の 0/1 テキスト)

## 7.5 フォント管理 (fonts.ts)

◆ `loadFont(font: FontDef): Promise<void>`

`FontDef.url` が指す TTF/OTF を `FontFace` で読み込み、`document.fonts` に登録する。同梱フォント用。

◆ `registerCustomFont(file: File): Promise<FontDef>`

ユーザーがアップロードした `File` を `FontFace` 経由で登録し、ID を付与した `FontDef` を返す。以降の `fontId` 指定で利用可能になる。

## 7.6 共有リンク (share.ts)

◆ `encodeStateToURL(state: AppState): string`

`overrides` を除く `AppState` を JSON 化し、base64url で圧縮した文字列を `?s=` クエリに載せた URL を返す。

◆ `decodeStateFromURL(): Partial<AppState> | null`

現在の URL から `?s=` を読み出し、復元した `AppState` を返す。不正な値や欠損フィールドは無視し、欠けた箇所は既定値で補完される。

### ▷ `overrides` を共有しない理由

ピクセル編集結果 (`overrides`) はサイズが大きく URL に載せづらいため、共有リンクには含めない。共有先で再生成するか、JSON エクスポート機能 (`OutputPanel` のダウンロード) を用いる。

## 7.7 React Hooks

◆ `useAppState(initial: AppState): [AppState, Dispatch<Action>]`

`useReducer` を用いた状態管理。サポートするアクション: `setText`, `setFontId`, `setSize`, `setWidth`, `setHeight`, `setFontSize`, `setThreshold`, `setOffsetX`, `setOffsetY`, `setBold`, `setFormat`, `setOverride`, `clearOverride`, `clearAllOverrides`, `setPreviewGridStep`, `setFreeSize`, `setFreeMatrix`, `setFreePixel`, `setFreeGridStep`, `clearFree`, `replaceState`。 `setFreePixel` は (x, y, value) を取る 1 セル単位の原子更新であり、マウスドラッグ描画中の `stale closure` による取り零しを避けるために用いる。

◆ `useGlyphs(state: AppState): Glyph[]`

`text`, `fontId`, 幾何パラメータ、`overrides` の変化にのみ反応して再計算するメモ化ラップ。

◆ `usePersist(state: AppState)`

`AppState` を `localStorage["font-to-bin.state.v1"]` に自動保存する副作用フック。`overrides` は除外する。

## 8 実行モデル

### 8.1 起動シーケンス

1. index.html が src/main.tsx を読み込み、React ルートを #root にマウント。
2. App.tsx が初期 AppState を決定する。
  - URL に ?s=... があれば decodeStateFromURL() を使用して復元。
  - なければ LocalStorage から persistedInitial() で読み込み。
  - どちらも無効な場合は DEFAULT\_STATE を採用。
3. useGlyphs が初期テキストをラスタライズし、formatOutput の結果が OutputPanel に反映される。
4. usePersist が以後の AppState 変化を LocalStorage に書き込み続ける。

### 8.2 ラスタライズの詳細

1. <canvas> を width × height で作成。
2. 背景を白でクリアし、fillStyle="black" で文字を描画。
  - ctx.font = '\$sizepx \$fontFamily' (太字時は bold を前置)。
  - ctx.textBaseline = "top"。
  - fillText(char, offsetX, offsetY)。
3. getImageData(0,0,width,height) で RGBA 配列を取得。
4. 各ピクセルのアルファチャンネル  $\alpha$  と閾値  $\tau$  を比較し、 $\alpha \geq \tau$  を 1、それ以外を 0 とする。
5. 結果を Matrix (matrix[row][col]) に格納。

### 8.3 ビットパック規則

#### 8.3.1 行方向パック (bitpack-row)

- 各行を左 → 右に走査し、8bit 単位で 1 バイトを構成。
- bitOrder="msb": 左端ピクセルが bit7。
- bitOrder="lsb": 左端ピクセルが bit0。
- padByte=true 時、行終端が 8bit 境界でなければ下位 (または上位) を 0 埋め。

#### 8.3.2 列方向パック (bitpack-col)

- 各列を上 → 下に走査し、縦 8bit を 1 バイトに格納。OLED SSD1306 の horizontal addressing mode と整合。
- bitOrder="msb": 上端ピクセルが bit7。
- bitOrder="lsb": 上端ピクセルが bit0 (SSD1306 既定)。

### 8.4 状態遷移 (Reducer)

useAppState の Reducer は次の不変条件を守る。

- width / height / size の変更時、既存の overrides は次元が一致しないため clearAllOverrides 扱い。
- text の変更時、文字数が減少すれば末尾の overrides を破棄。
- fontId の変更時、推奨サイズがあれば size を合わせる (ユーザーが明示的に変更中でない場合のみ)。

- `replaceState`（共有リンク適用や読込）は `overrides` を空にする。

## 8.5 永続化

- キー: `font-to-bin.state.v1`（スキーマ変更時は `.v2` へ上げる）。
- 保存対象: `AppState` から `overrides` を除いた部分。
- 復元時のバリデーション: 型不一致・未知フィールドは無視し、欠損は既定値で補完。
- ユーザーはヘッダーの「設定リセット」で `LocalStorage` を削除可能。

## 8.6 共有リンクのラウンドトリップ

1. 「URL 共有」ボタン押下で `encodeStateToURL(state)` を実行。
2. `JSON.stringify` → `TextEncoder` → `base64url` 変換。
3. 現在の URL の `?s=` を差し替え、クリップボードへコピー。
4. 開いた側では起動時に `decodeStateFromURL()` が復元。展開失敗時は既定値へフォールバック。

## 8.7 パフォーマンス考慮

- `useGlyphs` は `text + geometry + overrides` のハッシュキー変化時のみ再実行。
- フォーマッタは文字数  $N$ 、幅  $W$ 、高さ  $H$  に対して  $O(NWH)$ 。典型値 ( $N = 8, W = H = 16$ ) では数百  $\mu\text{s}$ 。
- ピクセルエディタは 1 グリフあたり  $W \times H$  ノードで描画するが、SVG の仮想 DOM に任せており数ミリ秒以内で再描画可能。

## 8.8 デプロイ

- `main` ブランチへの `push` で GitHub Actions が起動。
- `actions/setup-node@v4` → `npm ci` → `npm run build` → `actions/upload-pages-artifact` → `actions/deploy-pages`。
- ビルド後、`dist/index.html` を `404.html` にコピーして SPA の任意パス直打ちに対応。

### ⚠ 初回のみ必要な設定

リポジトリの **Settings** → **Pages** → **Source** を GitHub Actions に設定する必要がある（`gh-pages` ブランチは使わない）。